

Functional Programming

Radu Răzvan Slăvescu

Technical University of Cluj-Napoca
Department of Computer Science

Outline

Anonymous functions: Lambda expressions

List comprehensions

Generators

Comprehensions=Generators+Guards

Some basic Higher Order Functions and Combinators

Combinators

Anonymous functions

	No name	Name
Data	1	a=1
Functions	?	inc x = x+1

Example (Lambda expressions)

Expression $\lambda x. x + x$ is called "lambda expression" and denotes a **nameless function** mapping a number into its double. It consists of the symbol λ , its parameter(s) (here, just x), a dot and the body (computations to be performed; here, $x + x$).

Example (Haskell lambdas: $\backslash$ for λ , $\rightarrow$ for the dot)

```
Main> (\ x -> x + x) 3
```

When can we benefit from lambda expressions?

Example (Avoid "wasting" function names)

```
squares n = map f [n, n+1, n+2] where  
  f x = x*x
```

could be defined in a simpler manner:

```
squares n = map (\ x -> x*x) [n, n+1, n+2]
```

When can we benefit from lambda expressions?

Example (Functions sent as arguments)

```
isord [] _ = True
isord [x] _ = True
isord (x:y:ys) ordrel =
    (ordrel x y) && isord (y:ys) ordrel
```

```
Main> isord ["Ro", "Pl", "Me", "Dk"] (\ x y -> x>y)
True
```

```
Main> isord ["Dk", "Pl", "Ro", "Me"] (\ x y -> x<y)
False
```

Remark

The lambda expression gives decides the ordering criterion.

The behavior of `isord` is **parameterized**.

When can we benefit from lambda expressions?

Example (Functions which return functions as results)

```
const1 :: a -> b -> a  
const1 n _ = n
```

```
const2 :: a -> b -> a  
const2 n = \ _ -> n
```

Remark

`const2` is more natural: the call `const2 n` **creates** and **returns** a function `\ _ -> n` which does the following things: (1) waits for an argument; (2) consumes it and (3) returns `n`.

When can we benefit from lambda expressions?

Example (Formal meaning of currying-based functions)

`subtract x y = x - y`

means

`subtract = \ x -> (\ y -> x - y)`

Remark

Now it should be clear that `subtract 1` waits for an argument and subtracts it from 1.

Ranges

A **range** is used for **enumerating** list elements.

Example

```
Main> [1..6]  
[1, 2, 3, 4, 5, 6]
```

Example

```
Main> [6..1]  
[]
```


Ranges

Example

```
Main> [2,4..10]  
[2,4,6,8,10]
```

Example

```
Main> [6,5..1]  
[6,5,4,3,2,1]
```

Remark

These generators build an arithmetic progression from one initial limit up to a final limit and with a given increment.

Ranges

Example (A Bottomless Coin Sack)

```
Main> take 5 [1..]  
[1,2,3,4,5]
```

Example (Boolean values)

```
Main> take 3 [False .. ]  
[False,True]
```

```
Main> take 3 [False, False .. ]  
[False,False,False]
```

List comprehensions

Definition

List comprehension = **list generators** + **guards** (filtering tests)

Example

```
Main> [ x | x <- [1..10], odd x]  
[1,3,5,7,9]
```

Components

1. element expression: `x`
2. generators: `x <- [1..10]`
3. guards: `odd x`

List comprehensions

Example

```
Main> [ 2*x | x <- [1..5], odd x]
[2,6,10]
```

Example

```
Main> [(x,x*x) | x <- [1..4]]
[(1,1),(2,4),(3,9),(4,16)]
```

Example

```
Main> [ "Go!" | x <- [1..3]]
["Go!","Go!","Go!"]
```

List comprehensions

Example

```
Main> [(b,g) | b <- ["Arnold","Brad"],  
              g <- ["Xenia", "Yvonne", "Zeta"]]  
  
[("Arnold","Xenia"), ("Arnold","Yvonne"),  
 ("Arnold","Zeta"), ("Brad","Xenia"),  
 ("Brad","Yvonne"), ("Brad","Zeta")]
```

Remark

Rightmost generator = innermost loop

List comprehensions

Example

```
Main> [[1..n] | n <- [1..4]]  
[[1],[1,2],[1,2,3],[1,2,3,4]]
```

Example

```
Main> take 4 [ [(i,j) | i <- [1..j]] | j <- [1..]]  
[(1,1)],  
[(1,2),(2,2)],  
[(1,3),(2,3),(3,3)],  
[(1,4),(2,4),(3,4),(4,4)]]
```

List comprehensions - your turn

Example

```
Main> [x | x <- [1..5], odd x]  
[1,3,5]
```

Example

```
Main> [odd x | x <- [1..5]]  
[True, False, True, False, True]
```

List comprehensions - your turn

Example

```
Main> [ x | x:xs <- ["Extremly", "Lumionous",  
                    "Marvel"]]
"ELM"
```

Example

```
Main> ["Romania" | n <- [1,2]] ++  
      ["Ole" | n <- [1..3]]  
["Romania", "Romania", "Ole", "Ole", "Ole"]
```


List comprehensions - where besides Haskell?

Example (Python)

```
[(x,x*x) for x in range(1,5)]  
[(1, 1), (2, 4), (3, 9), (4, 16)]
```

Example (Python)

```
[(x,x*x) for x in range(1,5) if (x%2) == 0]  
[(2, 4), (4, 16)]
```

Example (Python)

```
[(x,y) for x in ["A","B"] for y in ["X", "Y", "Z"]]  
[('A', 'X'), ('A', 'Y'), ('A', 'Z'), ('B', 'X'),  
 ('B', 'Y'), ('B', 'Z')]
```

List comprehensions - your turn

Example

```
innpro xs ys =  
  let  
    li = length xs - 1  
  in  
    sum [ (xs !! n) * (ys !! n) | n <- [0..li]]
```

```
Main> innpro [1,3,5] [2,4,6]  
44
```

List comprehensions - your turn

Example

```
caupro xs ys =  
  let  
    innpro xs ys = sum (zipWith (*) xs ys)  
  in  
    [ innpro (take k xs) (reverse (take k ys)) |  
      k <- [1..] ]
```

```
Main> take 5 (caupro [1,1,2,1,3] [1,2,3,2,3])  
[1,3,7,10,16]
```

List comprehensions - why?

Example

```
apN :: [[a]] -> [a]
apN xss = [x | xs <- xss, x <- xs]

Main> apN [[".rs", ".lt"],
           [".fr", ".at", ".ro"],
           [".fo"]]
[".rs", ".lt", ".fr", ".at", ".ro", ".fo"]
```

List comprehensions - why?

Example

```
qs :: Ord a => [a] -> [a]
```

```
qs [] = []
```

```
qs (x:xs) = qs smalls ++ [x] ++ qs bigs
```

```
    where
```

```
        smalls = [s | s<-xs, s<=x]
```

```
        bigs = [b | b<-xs, b>x]
```

A better version

The prior version goes through the list twice.

Example

```
my_split _ [] smalls bigs = (smalls,bigs)
my_split x (y:ys) smalls bigs
  | y < x = my_split x ys (y:smalls) bigs
  | otherwise = my_split x ys smalls (y:bigs)

qs [] = []
qs (x:xs) = qs smalls ++ [x] ++ qs bigs
  where
    (smalls,bigs) = my_split x xs [] []
```

Generating permutations

Example

```
elim :: Eq a => a -> [a] -> [a]
elim _ [] = []
elim e (x:xs) | e == x    = elim e xs
               | otherwise = x:elim e xs

pr :: Eq t => [t] -> [[t]]
pr [] = [[]]
pr xs = [ a:p | a <- xs ,  p <- pr (elim a xs) ]
```

The higher-order function *map*

Definition (Map)

It applies a given function on every element of a list.

Example (Using map)

```
Main> map (+1) [1,2,3]  
[2,3,4]
```

Example (Defining map using list comprehensions)

```
mapGen :: (a->b) -> [a] -> [b]  
mapGen f xs = [ f x | x <- xs]
```


The higher-order function *filter*

Definition (Filter)

It selects the elements of a list satisfying a given predicate.

Example (Using filter)

```
Main> filter odd [1..5]
[1,3,5]
```

Example (Defining filter using list comprehensions)

```
filterGen :: (a -> Bool) -> [a] -> [a]
filterGen p xs = [ x | x <- xs, p x]
```

The higher-order function *foldr*

Example (Sum)

```
mysum [] = 0  
mysum (x:xs) = x + mysum xs
```

Example (Product)

```
myprod [] = 1  
myprod (x:xs) = x * myprod xs
```

Example (And)

```
myand [] = True  
myand (x:xs) = x && myand xs
```

The higher-order function *foldr*

Many functions are defined using the following pattern:

$$\begin{aligned}f [] &= v \\f (x : xs) &= x \oplus f xs\end{aligned}$$

Function `foldr :: (a -> b -> b) -> b -> [a] -> b` incorporates this pattern, with the $\oplus$ operation and the `v` value as arguments/parameters, and **adds it to the language**.

Example (New definitions for `mysum` and `myand`)

```
mynewsum = foldr (+) 0
```

```
mynewand = foldr (&&) True
```

Combinators

Problem

Write a curried function which decrements its argument by 1.

Example (A wrong approach)

```
((-) 1)
```

```
Main> ((-) 1) 9  
-8
```

Explanation

Arguments are taken from left to right. What to do?

The C combinator

Example (Solution: C combinator, aka `flip` in Haskell)

```
combC f x y = f y x
```

```
Main> combC (-) 1 9  
8
```

Explanation

`combC` swaps the (first two) arguments of `f`

Definition (Combinator)

Combinator = a function with no free variables.

Though we'll use this name only for a couple of such functions.

The K combinator

Explanation

The K combinator creates a constant function.

Example (Combinator K)

```
combK n x = n
```

```
Main> combK 3 "anything"  
3
```

Combinators and the Point-Free style

Grade conversion

Let us consider two countries, C1 and C2, with grade scales $[mi1, ma1]$ and $[mi2, ma2]$ respectively. (e.g., $[0..4]$ for US GPA and $[1..20]$ France). In country C2 my average is m . What is its corresponding value in country C1?

Example (Scale conversion: to, from, mark)

```
scaleMark mi1 ma1 mi2 ma2 m =  
  mi1 + (m-mi2) / (ma2-mi2) * (ma1-mi1)
```

Combinators and the Point-Free style

Example (Scale conversion: to, from, mark)

```
scaleMark mi1 ma1 mi2 ma2 m =  
  mi1 + (m-mi2) / (ma2-mi2) * (ma1-mi1)
```

```
toGPA = scaleMark 0.0 4.0
```

```
mix5 f p1 p2 p3 p4 p5 = f p3 p4 p1 p2 p5  
franceToAnywhere = mix5 scaleMark 1 20
```

Remark

Functions `toGPA` and `franceToAnywhere` are written in **point-free style**. Their arguments are "hidden".

Summary

- ▶ lambda expressions are nameless functions
- ▶ list comprehensions (generators + guards) allow easy list manipulation
- ▶ `map`, `filter` and `foldr` focus on lists instead of individual elements
- ▶ lambda expressions, combinators, and higher-order functions help us writing code in a compact manner